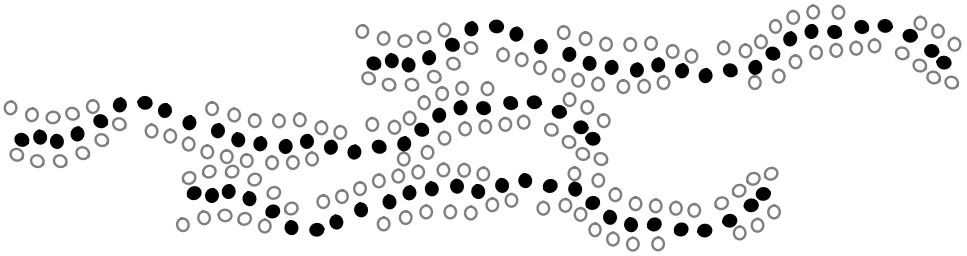
Chapter 13 Text Processing



Contents

13.1 Abundance of Digitized Text 582
13.1.1 Notations for Strings and the Python str Class 583
13.2 Pattern-Matching Algorithms 584
13.2.1 Brute Force 584
13.2.2 The Boyer-Moore Algorithm 586
13.2.3 The Knuth-Morris-Pratt Algorithm 590
13.3 Dynamic Programming 594
13.3.1 Matrix Chain-Product 594
13.3.2 DNA and Text Sequence Alignment 597
13.4 Text Compression and the Greedy Method 601
13.4.1 The Huffman Coding Algorithm 602
13.4.2 The Greedy Method 603
13.5 Tries 604
13.5.1 Standard Tries 604
13.5.2 Compressed Tries 608
13.5.3 Suffix Tries 610
13.5.4 Search Engine Indexing 612
13.6 Exercises 613

13.1 Abundance of Digitized Text

Despite the wealth of multimedia information, text processing remains one of the dominant functions of computers. Computer are used to edit, store, and display documents, and to transport documents over the Internet. Furthermore, digital systems are used to archive a wide range of textual information, and new data is being generated at a rapidly increasing pace. A large corpus can readily surpass a petabyte of data (which is equivalent to a thousand terabytes, or a million gigabytes). Common examples of digital collections that include textual information are:

- Snapshots of the World Wide Web, as Internet document formats HTML and XML are primarily text formats, with added tags for multimedia content
- All documents stored locally on a user's computer
- Email archives
- Customer reviews
- Compilations of status updates on social networking sites such as Facebook
- Feeds from microblogging sites such as Twitter and Tumblr

These collections include written text from hundreds of international languages. Furthermore, there are large data sets (such as DNA) that can be viewed computationally as "strings" even though they are not language.

In this chapter we explore some of the fundamental algorithms that can be used to efficiently analyze and process large textual data sets. In addition to having interesting applications, text-processing algorithms also highlight some important algorithmic design patterns.

We begin by examining the problem of searching for a pattern as a substring of a larger piece of text, for example, when searching for a word in a document. The pattern-matching problem gives rise to the ***brute-force method***, which is often inefficient but has wide applicability.

Next, we introduce an algorithmic technique known as ***dynamic programming***, which can be applied in certain settings to solve a problem in polynomial time that appears at first to require exponential time to solve. We demonstrate the application on this technique to the problem of finding partial matches between strings that may be similar but not perfectly aligned. This problem arises when making suggestions for a misspelled word, or when trying to match related genetic samples.

Because of the massive size of textual data sets, the issue of compression is important, both in minimizing the number of bits that need to be communicated through a network and to reduce the long-term storage requirements for archives. For text compression, we can apply the ***greedy method***, which often allows us to approximate solutions to hard problems, and for some problems (such as in text compression) actually gives rise to optimal algorithms.

Finally, we examine several special-purpose data structures that can be used to better organize textual data in order to support more efficient run-time queries.

13.1.1 Notations for Strings and the Python str Class

We use character strings as a model for text when discuss algorithms for text processing. Character strings can come from a wide variety of sources, including scientific, linguistic, and Internet applications. Indeed, the following are examples of such strings:

$$S = \texttt{"CGTAAACTGCTTTAATCAAACGC"}$$
$$T = \texttt{"http://www.wiley.com"}$$

The first string, S, comes from DNA applications, and the second string, T, is the Internet address (URL) for the publisher of this book. We refer to Appendix A for an overview of the operations supported by Python's **str** class.

To allow fairly general notions of a string in our algorithm descriptions, we only assume that characters of a string come from a known ***alphabet***, which we denote as Σ. For example, in the context of DNA, there are four symbols in the standard alphabet, $\Sigma = \{A, C, G, T\}$. This alphabet Σ can, of course, be a subset of the ASCII or Unicode character sets, but it could also be something more general. Although we assume that an alphabet has a fixed finite size, denoted as $|\Sigma|$, that size can be nontrivial, as with Python's treatment of the Unicode alphabet, which allows for more than a million distinct characters. We therefore consider the impact of $|\Sigma|$ in our asymptotic analysis of text-processing algorithms.

Several string-processing operations involve breaking large strings into smaller strings. In order to be able to speak about the pieces that result from such operations, we will rely on Python's ***indexing*** and ***slicing*** notations. For the sake of notation, we let S denote a string of length n. In that case, we let S[j] refer to the character at index j for $0 \leq j \leq n-1$. We let notation S[j:k] for $0 \leq j \leq k \leq n$ denote the slice (or ***substring***) of S consisting of characters S[j] up to and including S[k−1], but not S[k]. By this definition, note that substring S[j : j + m] has length m and that substring S[j:j] is trivially the ***null string***, having length 0. In accordance with Python conventions, the substring S[j:k] is also the null string when $k < j$.

In order to distinguish some special kinds of substrings, let us refer to any substring of the form S[0:k] for $0 \leq k \leq n$ as a ***prefix*** of S; such a prefix results in Python when the first index is omitted from slice notation, as in S[:k]. Similarly, any substring of the form S[j:n] for $0 \leq j \leq n$ is a ***suffix*** of S; such a suffix results in Python when the second index is omitted from slice notation, as in S[j:]. For example, if we again take S to be the string of DNA given above, then `"CGTAA"` is a prefix of S, `"CGC"` is a suffix of S, and `"C"` is both a prefix and suffix of S. Note that the null string is a prefix and a suffix of any string.

13.2 Pattern-Matching Algorithms

In the classic ***pattern-matching*** problem, we are given a ***text*** string T of length n and a ***pattern*** string P of length m, and want to find whether P is a substring of T. If so, we may want to find the lowest index j within T at which P begins, such that T[j:j+m] equals P, or perhaps to find *all* indices of T at which pattern P begins.

The pattern-matching problem is inherent to many behaviors of Python's **str** class, such as P in T, T.find(P), T.index(P), T.count(P), and is a subtask of more complex behaviors such as T.partition(P), T.split(P), and T.replace(P, Q).

In this section, we present three pattern-matching algorithms (with increasing levels of difficulty). For simplicity, we model the outward semantics of our functions upon the find method of the string class, returning the lowest index at which the pattern begins, or -1 if the pattern is not found.

13.2.1 Brute Force

The ***brute-force*** algorithmic design pattern is a powerful technique for algorithm design when we have something we wish to search for or when we wish to optimize some function. When applying this technique in a general situation, we typically enumerate all possible configurations of the inputs involved and pick the best of all these enumerated configurations.

In applying this technique to design a brute-force pattern-matching algorithm, we derive what is probably the first algorithm that we might think of for solving the problem—we simply test all the possible placements of P relative to T. An implementation of this algorithm is shown in Code Fragment 13.1.

```
1  def find_brute(T, P):
2    """Return the lowest index of T at which substring P begins (or else -1)."""
3    n, m = len(T), len(P)              # introduce convenient notations
4    for i in range(n-m+1):             # try every potential starting index within T
5      k = 0                            # an index into pattern P
6      while k < m and T[i + k] == P[k]:        # kth character of P matches
7        k += 1
8      if k == m:                       # if we reached the end of pattern,
9        return i                       # substring T[i:i+m] matches P
10   return -1                          # failed to find a match starting with any i
```

Code Fragment 13.1: An implementation of brute-force pattern-matching algorithm.

Performance

The analysis of the brute-force pattern-matching algorithm could not be simpler. It consists of two nested loops, with the outer loop indexing through all possible starting indices of the pattern in the text, and the inner loop indexing through each character of the pattern, comparing it to its potentially corresponding character in the text. Thus, the correctness of the brute-force pattern-matching algorithm follows immediately from this exhaustive search approach.

The running time of brute-force pattern matching in the worst case is not good, however, because, for each candidate index in T, we can perform up to m character comparisons to discover that P does not match T at the current index. Referring to Code Fragment 13.1, we see that the outer **for** loop is executed at most $n-m+1$ times, and the inner **while** loop is executed at most m times. Thus, the worst-case running time of the brute-force method is $O(nm)$.

Example 13.1: *Suppose we are given the text string*

$$T = \texttt{"abacaabaccabacabaabb"}$$

and the pattern string

$$P = \texttt{"abacab"}$$

Figure 13.1 illustrates the execution of the brute-force pattern-matching algorithm on T and P.

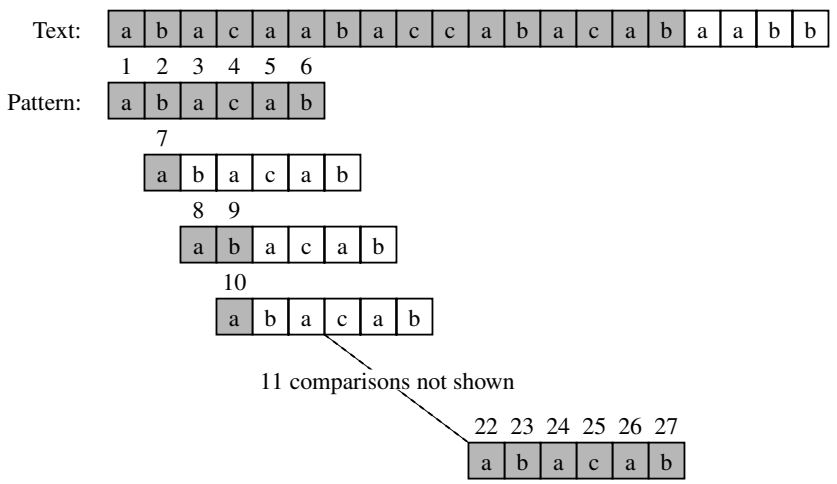


Figure 13.1: Example run of the brute-force pattern-matching algorithm. The algorithm performs 27 character comparisons, indicated above with numerical labels.

13.2.2 The Boyer-Moore Algorithm

At first, it might seem that it is always necessary to examine every character in T in order to locate a pattern P as a substring or to rule out its existence. But this is not always the case. The ***Boyer-Moore*** pattern-matching algorithm, which we study in this section, can sometimes avoid comparisons between P and a sizable fraction of the characters in T. In this section, we describe a simplified version of the original algorithm by Boyer and Moore.

The main idea of the Boyer-Moore algorithm is to improve the running time of the brute-force algorithm by adding two potentially time-saving heuristics. Roughly stated, these heuristics are as follows:

***Looking-Glass Heuristic*:** When testing a possible placement of P against T, begin the comparisons from the end of P and move backward to the front of P.

***Character-Jump Heuristic*:** During the testing of a possible placement of P within T, a mismatch of text character T[i]=c with the corresponding pattern character P[k] is handled as follows. If c is not contained anywhere in P, then shift P completely past T[i] (for it cannot match any character in P). Otherwise, shift P until an occurrence of character c in P gets aligned with T[i].

We will formalize these heuristics shortly, but at an intuitive level, they work as an integrated team. The looking-glass heuristic sets up the other heuristic to allow us to avoid comparisons between P and whole groups of characters in T. In this case at least, we can get to the destination faster by going backwards, for if we encounter a mismatch during the consideration of P at a certain location in T, then we are likely to avoid lots of needless comparisons by significantly shifting P relative to T using the character-jump heuristic. The character-jump heuristic pays off big if it can be applied early in the testing of a potential placement of P against T. Figure 13.2 demonstrates a few simple applications of these heuristics.

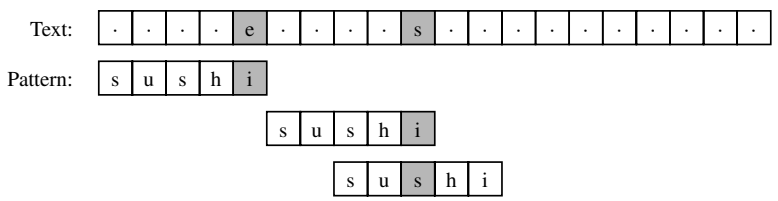


Figure 13.2: A simple example demonstrating the intuition of the Boyer-Moore pattern-matching algorithm. The original comparison results in a mismatch with character e of the text. Because that character is nowhere in the pattern, the entire pattern is shifted beyond its location. The second comparison is also a mismatch, but the mismatched character s occurs elsewhere in the pattern. The pattern is next shifted so that its last occurrence of s is aligned with the corresponding s in the text. The remainder of the process is not illustrated in this figure.

The example of Figure 13.2 is rather basic, because it only involves mismatches with the last character of the pattern. More generally, when a match is found for that last character, the algorithm continues by trying to extend the match with the second-to-last character of the pattern in its current alignment. That process continues until either matching the entire pattern, or finding a mismatch at some interior position of the pattern.

If a mismatch is found, and the mismatched character of the text does not occur in the pattern, we shift the entire pattern beyond that location, as originally illustrated in Figure 13.2. If the mismatched character occurs elsewhere in the pattern, we must consider two possible subcases depending on whether its last occurrence is before or after the character of the pattern that was aligned with the mismatched. Those two cases are illustrated in Figure 13.3.

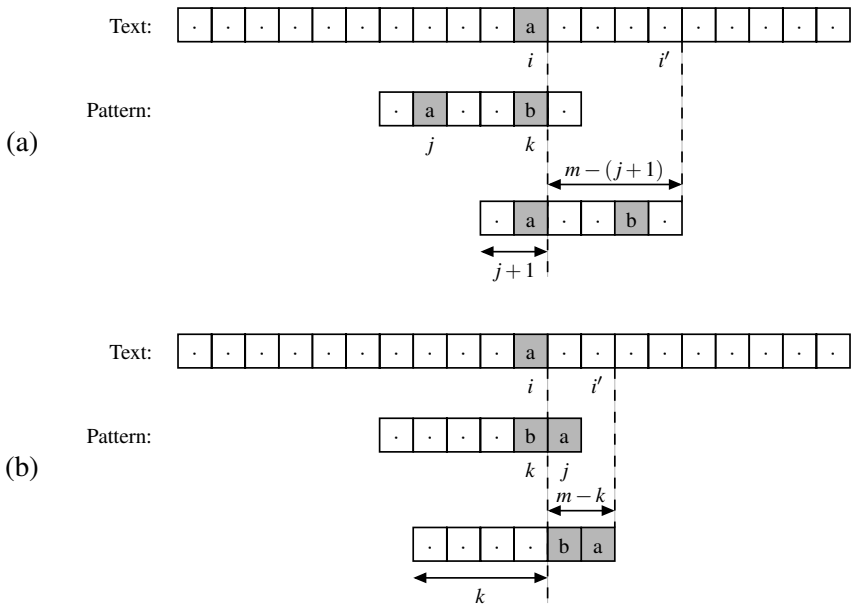


Figure 13.3: Additional rules for the character-jump heuristic of the Boyer-Moore algorithm. We let i represent the index of the mismatched character in the text, k represent the corresponding index in the pattern, and j represent the index of the last occurrence of $T[i]$ within the pattern. We distinguish two cases: (a) $j < k$, in which case we shift the pattern by $k - j$ units, and thus, index i advances by $m - (j+1)$ units; (b) $j > k$, in which case we shift the pattern by one unit, and index i advances by $m - k$ units.

In the case of Figure 13.3(b), we slide the pattern only one unit. It would be more productive to slide it rightward until finding another occurrence of mismatched character $T[i]$ in the pattern, but we do not wish to take time to search for

another occurrence. The efficiency of the Boyer-Moore algorithm relies on creating a lookup table that quickly determines where a mismatched character occurs elsewhere in the pattern. In particular, we define a function $\mathsf{last}(c)$ as

- If c is in P, $\mathsf{last}(c)$ is the index of the last (rightmost) occurrence of c in P. Otherwise, we conventionally define $\mathsf{last}(c) = -1$.

If we assume that the alphabet is of fixed, finite size, and that characters can be converted to indices of an array (for example, by using their character code), the last function can be easily implemented as a lookup table with worst-case $O(1)$-time access to the value $\mathsf{last}(c)$. However, the table would have length equal to the size of the alphabet (rather than the size of the pattern), and time would be required to initialize the entire table.

We prefer to use a hash table to represent the last function, with only those characters from the pattern occurring in the structure. The space usage for this approach is proportional to the number of distinct alphabet symbols that occur in the pattern, and thus $O(m)$. The expected lookup time remains independent of the problem (although the worst-case bound is $O(m)$). Our complete implementation of the Boyer-Moore pattern-matching algorithm is given in Code Fragment 13.2.

```
1  def find_boyer_moore(T, P):
2    """Return the lowest index of T at which substring P begins (or else -1)."""
3    n, m = len(T), len(P)                    # introduce convenient notations
4    if m == 0: return 0                      # trivial search for empty string
5    last = { }                               # build 'last' dictionary
6    for k in range(m):
7      last[ P[k] ] = k                       # later occurrence overwrites
8    # align end of pattern at index m-1 of text
9    i = m−1                                  # an index into T
10   k = m−1                                  # an index into P
11   while i < n:
12     if T[i] == P[k]:                       # a matching character
13       if k == 0:
14         return i                           # pattern begins at index i of text
15       else:
16         i −= 1                             # examine previous character
17         k −= 1                             # of both T and P
18     else:
19       j = last.get(T[i], −1)               # last(T[i]) is -1 if not found
20       i += m − min(k, j + 1)               # case analysis for jump step
21       k = m − 1                            # restart at end of pattern
22   return −1
```

Code Fragment 13.2: An implementation of the Boyer-Moore algorithm.

The correctness of the Boyer-Moore pattern-matching algorithm follows from the fact that each time the method makes a shift, it is guaranteed not to "skip" over any possible matches. For $\mathsf{last}(c)$ is the location of the ***last*** occurrence of c in P. In Figure 13.4, we illustrate the execution of the Boyer-Moore pattern-matching algorithm on an input string similar to Example 13.1.

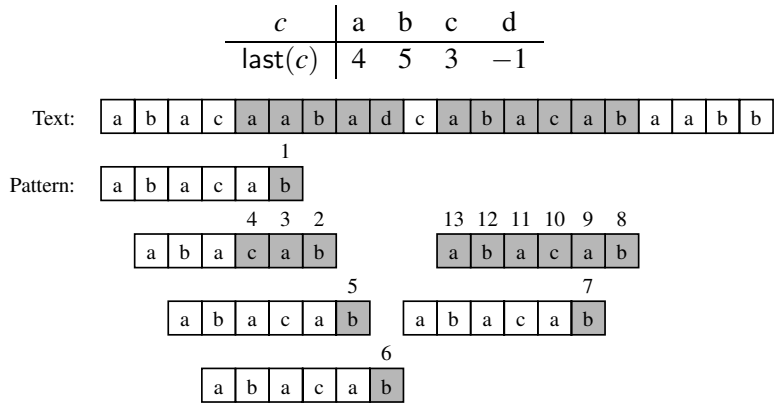


Figure 13.4: An illustration of the Boyer-Moore pattern-matching algorithm, including a summary of the $\mathsf{last}(c)$ function. The algorithm performs 13 character comparisons, which are indicated with numerical labels.

Performance

If using a traditional lookup table, the worst-case running time of the Boyer-Moore algorithm is $O(nm+|\Sigma|)$. Namely, the computation of the last function takes time $O(m+|\Sigma|)$, and the actual search for the pattern takes $O(nm)$ time in the worst case, the same as the brute-force algorithm. (With a hash table, the dependence on $|\Sigma|$ is removed.) An example of a text-pattern pair that achieves the worst case is

$$T = \overbrace{aaaaaa\cdots a}^{n}$$
$$P = b\overbrace{aa\cdots a}^{m-1}$$

The worst-case performance, however, is unlikely to be achieved for English text, for, in that case, the Boyer-Moore algorithm is often able to skip large portions of text. Experimental evidence on English text shows that the average number of comparisons done per character is 0.24 for a five-character pattern string.

We have actually presented a simplified version of the Boyer-Moore algorithm. The original algorithm achieves running time $O(n+m+|\Sigma|)$ by using an alternative shift heuristic to the partially matched text string, whenever it shifts the pattern more than the character-jump heuristic. This alternative shift heuristic is based on applying the main idea from the Knuth-Morris-Pratt pattern-matching algorithm, which we discuss next.

13.2.3 The Knuth-Morris-Pratt Algorithm

In examining the worst-case performances of the brute-force and Boyer-Moore pattern-matching algorithms on specific instances of the problem, such as that given in Example 13.1, we should notice a major inefficiency. For a certain alignment of the pattern, if we find several matching characters but then detect a mismatch, we ignore all the information gained by the successful comparisons after restarting with the next incremental placement of the pattern.

The Knuth-Morris-Pratt (or "KMP") algorithm, discussed in this section, avoids this waste of information and, in so doing, it achieves a running time of $O(n+m)$, which is asymptotically optimal. That is, in the worst case any pattern-matching algorithm will have to examine all the characters of the text and all the characters of the pattern at least once. The main idea of the KMP algorithm is to precompute self-overlaps between portions of the pattern so that when a mismatch occurs at one location, we immediately know the maximum amount to shift the pattern before continuing the search. A motivating example is shown in Figure 13.5.

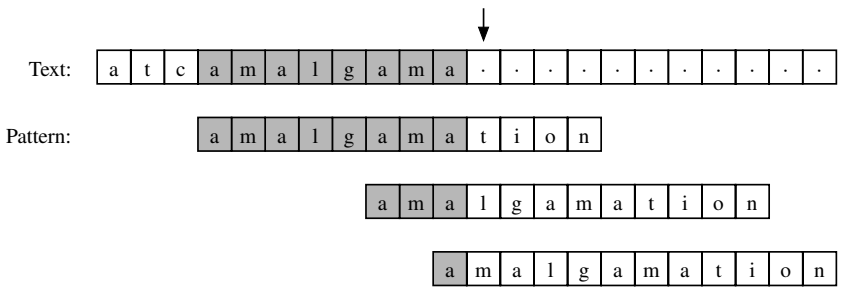


Figure 13.5: A motivating example for the Knuth-Morris-Pratt algorithm. If a mismatch occurs at the indicated location, the pattern could be shifted to the second alignment, without explicit need to recheck the partial match with the prefix `ama`. If the mismatched character is not an `l`, then the next potential alignment of the pattern can take advantage of the common `a`.

The Failure Function

To implement the KMP algorithm, we will precompute a ***failure function***, f, that indicates the proper shift of P upon a failed comparison. Specifically, the failure function $f(k)$ is defined as the length of the longest prefix of P that is a suffix of P[1:k+1] (note that we did ***not*** include P[0] here, since we will shift at least one unit). Intuitively, if we find a mismatch upon character P[k+1], the function $f(k)$ tells us how many of the immediately preceding characters can be reused to restart the pattern. Example 13.2 describes the value of the failure function for the example pattern from Figure 13.5.

Example 13.2: *Consider the pattern $P =$ "amalgamation" from Figure 13.5. The Knuth-Morris-Pratt (KMP) failure function, $f(k)$, for the string P is as shown in the following table:*

k	0	1	2	3	4	5	6	7	8	9	10	11
$P[k]$	a	m	a	l	g	a	m	a	t	i	o	n
$f(k)$	0	0	1	0	0	1	2	3	0	0	0	0

Implementation

Our implementation of the KMP pattern-matching algorithm is shown in Code Fragment 13.3. It relies on a utility function, compute_kmp_fail, discussed on the next page, to compute the failure function efficiently.

The main part of the KMP algorithm is its **while** loop, each iteration of which performs a comparison between the character at index j in T and the character at index k in P. If the outcome of this comparison is a match, the algorithm moves on to the next characters in both T and P (or reports a match if reaching the end of the pattern). If the comparison failed, the algorithm consults the failure function for a new candidate character in P, or starts over with the next index in T if failing on the first character of the pattern (since nothing can be reused).

```
1  def find_kmp(T, P):
2    """Return the lowest index of T at which substring P begins (or else -1)."""
3    n, m = len(T), len(P)                    # introduce convenient notations
4    if m == 0: return 0                      # trivial search for empty string
5    fail = compute_kmp_fail(P)               # rely on utility to precompute
6    j = 0                                    # index into text
7    k = 0                                    # index into pattern
8    while j < n:
9      if T[j] == P[k]:                       # P[0:1+k] matched thus far
10       if k == m - 1:                       # match is complete
11         return j - m + 1
12       j += 1                               # try to extend match
13       k += 1
14     elif k > 0:
15       k = fail[k-1]                        # reuse suffix of P[0:k]
16     else:
17       j += 1
18   return -1                                # reached end without match
```

Code Fragment 13.3: An implementation of the KMP pattern-matching algorithm. The compute_kmp_fail utility function is given in Code Fragment 13.4.

Constructing the KMP Failure Function

To construct the failure function, we use the method shown in Code Fragment 13.4, which is a "bootstrapping" process that compares the pattern to itself as in the KMP algorithm. Each time we have two characters that match, we set $f(j) = k+1$. Note that since we have $j > k$ throughout the execution of the algorithm, $f(k-1)$ is always well defined when we need to use it.

```
1  def compute_kmp_fail(P):
2    """Utility that computes and returns KMP 'fail' list."""
3    m = len(P)
4    fail = [0] * m                  # by default, presume overlap of 0 everywhere
5    j = 1
6    k = 0
7    while j < m:                    # compute f(j) during this pass, if nonzero
8      if P[j] == P[k]:              # k + 1 characters match thus far
9        fail[j] = k + 1
10       j += 1
11       k += 1
12     elif k > 0:                   # k follows a matching prefix
13       k = fail[k-1]
14     else:                         # no match found starting at j
15       j += 1
16   return fail
```

Code Fragment 13.4: An implementation of the compute_kmp_fail utility in support of the KMP pattern-matching algorithm. Note how the algorithm uses the previous values of the failure function to efficiently compute new values.

Performance

Excluding the computation of the failure function, the running time of the KMP algorithm is clearly proportional to the number of iterations of the **while** loop. For the sake of the analysis, let us define $s = j - k$. Intuitively, s is the total amount by which the pattern P has been shifted with respect to the text T. Note that throughout the execution of the algorithm, we have $s \leq n$. One of the following three cases occurs at each iteration of the loop.

- If $T[j] = P[k]$, then j and k each increase by 1, and thus, s does not change.
- If $T[j] \neq P[k]$ and $k > 0$, then j does not change and s increases by at least 1, since in this case s changes from $j-k$ to $j - f(k-1)$, which is an addition of $k - f(k-1)$, which is positive because $f(k-1) < k$.
- If $T[j] \neq P[k]$ and $k = 0$, then j increases by 1 and s increases by 1, since k does not change.

Thus, at each iteration of the loop, either j or s increases by at least 1 (possibly both); hence, the total number of iterations of the **while** loop in the KMP pattern-matching algorithm is at most $2n$. Achieving this bound, of course, assumes that we have already computed the failure function for P.

The algorithm for computing the failure function runs in $O(m)$ time. Its analysis is analogous to that of the main KMP algorithm, yet with a pattern of length m compared to itself. Thus, we have:

Proposition 13.3: *The Knuth-Morris-Pratt algorithm performs pattern matching on a text string of length n and a pattern string of length m in $O(n+m)$ time.*

The correctness of this algorithm follows from the definition of the failure function. Any comparisons that are skipped are actually unnecessary, for the failure function guarantees that all the ignored comparisons are redundant—they would involve comparing the same matching characters over again.

In Figure 13.6, we illustrate the execution of the KMP pattern-matching algorithm on the same input strings as in Example 13.1. Note the use of the failure function to avoid redoing one of the comparisons between a character of the pattern and a character of the text. Also note that the algorithm performs fewer overall comparisons than the brute-force algorithm run on the same strings (Figure 13.1).

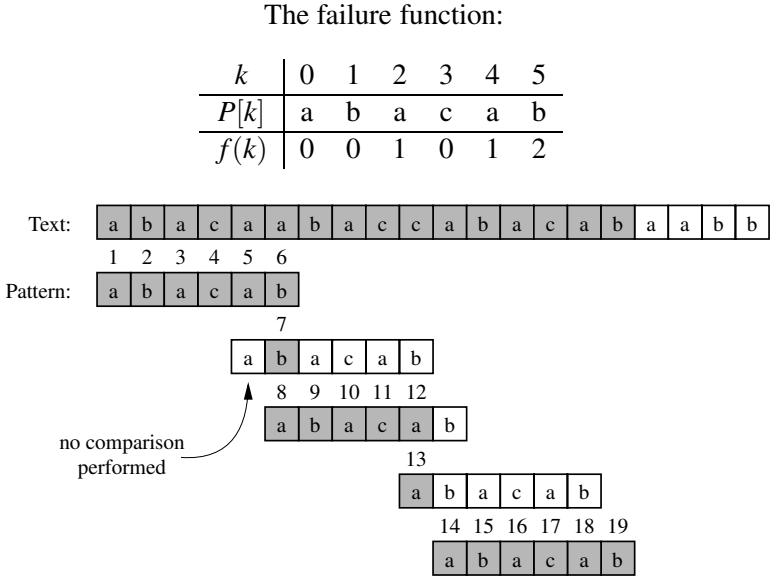


Figure 13.6: An illustration of the KMP pattern-matching algorithm. The primary algorithm performs 19 character comparisons, which are indicated with numerical labels. (Additional comparisons would be performed during the computation of the failure function.)

13.3 Dynamic Programming

In this section, we discuss the ***dynamic programming*** algorithm-design technique. This technique is similar to the divide-and-conquer technique (Section 12.2.1), in that it can be applied to a wide variety of different problems. Dynamic programming can often be used to take problems that seem to require exponential time and produce polynomial-time algorithms to solve them. In addition, the algorithms that result from applications of the dynamic programming technique are usually quite simple—often needing little more than a few lines of code to describe some nested loops for filling in a table.

13.3.1 Matrix Chain-Product

Rather than starting out with an explanation of the general components of the dynamic programming technique, we begin by giving a classic, concrete example. Suppose we are given a collection of n two-dimensional matrices for which we wish to compute the mathematical product

$$A = A_0 \cdot A_1 \cdot A_2 \cdots A_{n-1},$$

where A_i is a $d_i \times d_{i+1}$ matrix, for $i = 0, 1, 2, \ldots, n-1$. In the standard matrix multiplication algorithm (which is the one we will use), to multiply a $d \times e$-matrix B times an $e \times f$-matrix C, we compute the product, A, as

$$A[i][j] = \sum_{k=0}^{e-1} B[i][k] \cdot C[k][j].$$

This definition implies that matrix multiplication is associative, that is, it implies that $B \cdot (C \cdot D) = (B \cdot C) \cdot D$. Thus, we can parenthesize the expression for A any way we wish and we will end up with the same answer. However, we will not necessarily perform the same number of primitive (that is, scalar) multiplications in each parenthesization, as is illustrated in the following example.

Example 13.4: *Let B be a 2×10-matrix, let C be a 10×50-matrix, and let D be a 50×20-matrix. Computing $B \cdot (C \cdot D)$ requires $2 \cdot 10 \cdot 20 + 10 \cdot 50 \cdot 20 = 10400$ multiplications, whereas computing $(B \cdot C) \cdot D$ requires $2 \cdot 10 \cdot 50 + 2 \cdot 50 \cdot 20 = 3000$ multiplications.*

The ***matrix chain-product*** problem is to determine the parenthesization of the expression defining the product A that minimizes the total number of scalar multiplications performed. As the example above illustrates, the differences between parenthesizations can be dramatic, so finding a good solution can result in significant speedups.

Defining Subproblems

One way to solve the matrix chain-product problem is to simply enumerate all the possible ways of parenthesizing the expression for A and determine the number of multiplications performed by each one. Unfortunately, the set of all different parenthesizations of the expression for A is equal in number to the set of all different binary trees that have n leaves. This number is exponential in n. Thus, this straightforward ("brute-force") algorithm runs in exponential time, for there are an exponential number of ways to parenthesize an associative arithmetic expression.

We can significantly improve the performance achieved by the brute-force algorithm, however, by making a few observations about the nature of the matrix chain-product problem. The first is that the problem can be split into ***subproblems***. In this case, we can define a number of different subproblems, each of which is to compute the best parenthesization for some subexpression $A_i \cdot A_{i+1} \cdots A_j$. As a concise notation, we use $N_{i,j}$ to denote the minimum number of multiplications needed to compute this subexpression. Thus, the original matrix chain-product problem can be characterized as that of computing the value of $N_{0,n-1}$. This observation is important, but we need one more in order to apply the dynamic programming technique.

Characterizing Optimal Solutions

The other important observation we can make about the matrix chain-product problem is that it is possible to characterize an optimal solution to a particular subproblem in terms of optimal solutions to its subproblems. We call this property the ***subproblem optimality*** condition.

In the case of the matrix chain-product problem, we observe that, no matter how we parenthesize a subexpression, there has to be some final matrix multiplication that we perform. That is, a full parenthesization of a subexpression $A_i \cdot A_{i+1} \cdots A_j$ has to be of the form $(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$, for some $k \in \{i, i+1, \ldots, j-1\}$. Moreover, for whichever k is the correct one, the products $(A_i \cdots A_k)$ and $(A_{k+1} \cdots A_j)$ must also be solved optimally. If this were not so, then there would be a global optimal that had one of these subproblems solved suboptimally. But this is impossible, since we could then reduce the total number of multiplications by replacing the current subproblem solution by an optimal solution for the subproblem. This observation implies a way of explicitly defining the optimization problem for $N_{i,j}$ in terms of other optimal subproblem solutions. Namely, we can compute $N_{i,j}$ by considering each place k where we could put the final multiplication and taking the minimum over all such choices.

Designing a Dynamic Programming Algorithm

We can therefore characterize the optimal subproblem solution, $N_{i,j}$, as

$$N_{i,j} = \min_{i \le k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\},$$

where $N_{i,i} = 0$, since no work is needed for a single matrix. That is, $N_{i,j}$ is the minimum, taken over all possible places to perform the final multiplication, of the number of multiplications needed to compute each subexpression plus the number of multiplications needed to perform the final matrix multiplication.

Notice that there is a ***sharing of subproblems*** going on that prevents us from dividing the problem into completely independent subproblems (as we would need to do to apply the divide-and-conquer technique). We can, nevertheless, use the equation for $N_{i,j}$ to derive an efficient algorithm by computing $N_{i,j}$ values in a bottom-up fashion, and storing intermediate solutions in a table of $N_{i,j}$ values. We can begin simply enough by assigning $N_{i,i} = 0$ for $i = 0, 1, \ldots, n-1$. We can then apply the general equation for $N_{i,j}$ to compute $N_{i,i+1}$ values, since they depend only on $N_{i,i}$ and $N_{i+1,i+1}$ values that are available. Given the $N_{i,i+1}$ values, we can then compute the $N_{i,i+2}$ values, and so on. Therefore, we can build $N_{i,j}$ values up from previously computed values until we can finally compute the value of $N_{0,n-1}$, which is the number that we are searching for. A Python implementation of this ***dynamic programming*** solution is given in Code Fragment 13.5; we use techniques from Section 5.6 for representing a multidimensional table in Python.

```
1  def matrix_chain(d):
2    """d is a list of n+1 numbers such that size of kth matrix is d[k]-by-d[k+1].
3
4    Return an n-by-n table such that N[i][j] represents the minimum number of
5    multiplications needed to compute the product of Ai through Aj inclusive.
6    """
7    n = len(d) - 1                          # number of matrices
8    N = [[0] * n for i in range(n)]         # initialize n-by-n result to zero
9    for b in range(1, n):                   # number of products in subchain
10     for i in range(n-b):                  # start of subchain
11       j = i + b                           # end of subchain
12       N[i][j] = min(N[i][k]+N[k+1][j]+d[i]*d[k+1]*d[j+1] for k in range(i,j))
13   return N
```

Code Fragment 13.5: Dynamic programming algorithm for the matrix chain-product problem.

Thus, we can compute $N_{0,n-1}$ with an algorithm that consists primarily of three nested loops (the third of which computes the min term). Each of these loops iterates at most n times per execution, with a constant amount of additional work within. Therefore, the total running time of this algorithm is $O(n^3)$.

13.3.2 DNA and Text Sequence Alignment

A common text-processing problem, which arises in genetics and software engineering, is to test the similarity between two text strings. In a genetics application, the two strings could correspond to two strands of DNA, for which we want to compute similarities. Likewise, in a software engineering application, the two strings could come from two versions of source code for the same program, for which we want to determine changes made from one version to the next. Indeed, determining the similarity between two strings is so common that the Unix and Linux operating systems have a built-in program, named `diff`, for comparing text files.

Given a string $X = x_0x_1x_2\cdots x_{n-1}$, a ***subsequence*** of X is any string that is of the form $x_{i_1}x_{i_2}\cdots x_{i_k}$, where $i_j < i_{j+1}$; that is, it is a sequence of characters that are not necessarily contiguous but are nevertheless taken in order from X. For example, the string $AAAG$ is a subsequence of the string $CG\underline{A}T\underline{AA}TT\underline{G}AGA$.

The DNA and text similarity problem we address here is the ***longest common subsequence*** (LCS) problem. In this problem, we are given two character strings, $X = x_0x_1x_2\cdots x_{n-1}$ and $Y = y_0y_1y_2\cdots y_{m-1}$, over some alphabet (such as the alphabet $\{A,C,G,T\}$ common in computational genetics) and are asked to find a longest string S that is a subsequence of both X and Y. One way to solve the longest common subsequence problem is to enumerate all subsequences of X and take the largest one that is also a subsequence of Y. Since each character of X is either in or not in a subsequence, there are potentially 2^n different subsequences of X, each of which requires $O(m)$ time to determine whether it is a subsequence of Y. Thus, this brute-force approach yields an exponential-time algorithm that runs in $O(2^n m)$ time, which is very inefficient. Fortunately, the LCS problem is efficiently solvable using ***dynamic programming***.

The Components of a Dynamic Programming Solution

As mentioned above, the dynamic programming technique is used primarily for ***optimization*** problems, where we wish to find the "best" way of doing something. We can apply the dynamic programming technique in such situations if the problem has certain properties:

Simple Subproblems: There has to be some way of repeatedly breaking the global optimization problem into subproblems. Moreover, there should be a way to parameterize subproblems with just a few indices, like i, j, k, and so on.

Subproblem Optimization: An optimal solution to the global problem must be a composition of optimal subproblem solutions.

Subproblem Overlap: Optimal solutions to unrelated subproblems can contain subproblems in common.

Applying Dynamic Programming to the LCS Problem

Recall that in the LCS problem, we are given two character strings, X and Y, of length n and m, respectively, and are asked to find a longest string S that is a subsequence of both X and Y. Since X and Y are character strings, we have a natural set of indices with which to define subproblems—indices into the strings X and Y. Let us define a subproblem, therefore, as that of computing the value $L_{j,k}$, which we will use to denote the length of a longest string that is a subsequence of both prefixes $X[0:j]$ and $Y[0:k]$. This definition allows us to rewrite $L_{j,k}$ in terms of optimal subproblem solutions. This definition depends on which of two cases we are in. (See Figure 13.7.)

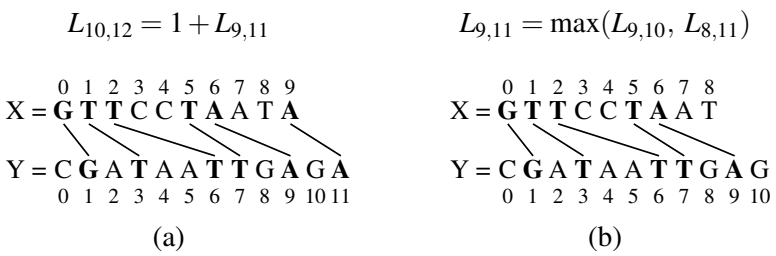


Figure 13.7: The two cases in the longest common subsequence algorithm for computing $L_{j,k}$: (a) $x_{j-1} = y_{k-1}$; (b) $x_{j-1} \neq y_{k-1}$.

- $x_{j-1} = y_{k-1}$. In this case, we have a match between the last character of $X[0:j]$ and the last character of $Y[0:k]$. We claim that this character belongs to a longest common subsequence of $X[0:j]$ and $Y[0:k]$. To justify this claim, let us suppose it is not true. There has to be some longest common subsequence $x_{a_1}x_{a_2}\ldots x_{a_c} = y_{b_1}y_{b_2}\ldots y_{b_c}$. If $x_{a_c} = x_{j-1}$ or $y_{b_c} = y_{k-1}$, then we get the same sequence by setting $a_c = j-1$ and $b_c = k-1$. Alternately, if $x_{a_c} \neq x_{j-1}$ and $y_{b_c} \neq y_{k-1}$, then we can get an even longer common subsequence by adding $x_{j-1} = y_{k-1}$ to the end. Thus, a longest common subsequence of $X[0:j]$ and $Y[0:k]$ ends with x_{j-1}. Therefore, we set

$$L_{j,k} = 1 + L_{j-1,k-1} \quad \text{if } x_{j-1} = y_{k-1}.$$

- $x_{j-1} \neq y_{k-1}$. In this case, we cannot have a common subsequence that includes both x_{j-1} and y_{k-1}. That is, we can have a common subsequence end with x_{j-1} or one that ends with y_{k-1} (or possibly neither), but certainly not both. Therefore, we set

$$L_{j,k} = \max\{L_{j-1,k}, L_{j,k-1}\} \quad \text{if } x_{j-1} \neq y_{k-1}.$$

We note that because slice $Y[0:0]$ is the empty string, $L_{j,0} = 0$ for $j = 0, 1, \ldots, n$; similarly, because slice $X[0:0]$ is the empty string, $L_{0,k} = 0$ for $k = 0, 1, \ldots, m$.

The LCS Algorithm

The definition of $L_{j,k}$ satisfies subproblem optimization, for we cannot have a longest common subsequence without also having longest common subsequences for the subproblems. Also, it uses subproblem overlap, because a subproblem solution $L_{j,k}$ can be used in several other problems (namely, the problems $L_{j+1,k}$, $L_{j,k+1}$, and $L_{j+1,k+1}$). Turning this definition of $L_{j,k}$ into an algorithm is actually quite straightforward. We create an $(n+1) \times (m+1)$ array, L, defined for $0 \leq j \leq n$ and $0 \leq k \leq m$. We initialize all entries to 0, in particular so that all entries of the form $L_{j,0}$ and $L_{0,k}$ are zero. Then, we iteratively build up values in L until we have $L_{n,m}$, the length of a longest common subsequence of X and Y. We give a Python implementation of this algorithm in Code Fragment 13.6.

```
1  def LCS(X, Y):
2    """Return table such that L[j][k] is length of LCS for X[0:j] and Y[0:k]."""
3    n, m = len(X), len(Y)                    # introduce convenient notations
4    L = [[0] * (m+1) for k in range(n+1)]    # (n+1) x (m+1) table
5    for j in range(n):
6      for k in range(m):
7        if X[j] == Y[k]:                     # align this match
8          L[j+1][k+1] = L[j][k] + 1
9        else:                                # choose to ignore one character
10         L[j+1][k+1] = max(L[j][k+1], L[j+1][k])
11   return L
```

Code Fragment 13.6: Dynamic programming algorithm for the LCS problem.

The running time of the algorithm of the LCS algorithm is easy to analyze, for it is dominated by two nested **for** loops, with the outer one iterating n times and the inner one iterating m times. Since the if-statement and assignment inside the loop each requires $O(1)$ primitive operations, this algorithm runs in $O(nm)$ time. Thus, the dynamic programming technique can be applied to the longest common subsequence problem to improve significantly over the exponential-time brute-force solution to the LCS problem.

The LCS function of Code Fragment 13.6 computes the length of the longest common subsequence (stored as $L_{n,m}$), but not the subsequence itself. Fortunately, it is easy to extract the actual longest common subsequence if given the complete table of $L_{j,k}$ values computed by the LCS function. The solution can be reconstructed back to front by reverse engineering the calculation of length $L_{n,m}$. At any position $L_{j,k}$, if $x_j = y_k$, then the length is based on the common subsequence associated with length $L_{j-1,k-1}$, followed by common character x_j. We can record x_j as part of the sequence, and then continue the analysis from $L_{j-1,k-1}$. If $x_j \neq y_k$,

then we can move to the larger of $L_{j,k-1}$ and $L_{j-1,k}$. We continue this process until reaching some $L_{j,k} = 0$ (for example, if j or k is 0 as a boundary case). A Python implementation of this strategy is given in Code Fragment 13.7. This function constructs a longest common subsequence in $O(n+m)$ additional time, since each pass of the **while** loop decrements either j or k (or both). An illustration of the algorithm for computing the longest common subsequence is given in Figure 13.8.

```
1  def LCS_solution(X, Y, L):
2    """Return the longest common substring of X and Y, given LCS table L."""
3    solution = [ ]
4    j,k = len(X), len(Y)
5    while L[j][k] > 0:                    # common characters remain
6      if X[j−1] == Y[k−1]:
7        solution.append(X[j−1])
8        j −= 1
9        k −= 1
10     elif L[j−1][k] >= L[j][k−1]:
11       j −=1
12     else:
13       k −= 1
14   return ''.join(reversed(solution))    # return left-to-right version
```

Code Fragment 13.7: Reconstructing the longest common subsequence.

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1	1	1	1	1
2	0	0	1	1	2	2	2	2	2	2	2	2	2
3	0	0	1	1	2	2	2	3	3	3	3	3	3
4	0	1	1	1	2	2	2	3	3	3	3	3	3
5	0	1	1	1	2	2	2	3	3	3	3	3	3
6	0	1	1	1	2	2	2	3	4	4	4	4	4
7	0	1	1	2	2	3	3	3	4	4	5	5	5
8	0	1	1	2	2	3	4	4	4	4	5	5	6
9	0	1	1	2	3	3	4	5	5	5	5	5	6
10	0	1	1	2	3	4	4	5	5	5	6	6	6

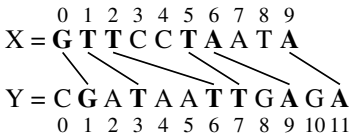


Figure 13.8: Illustration of the algorithm for constructing a longest common subsequence from the array L. A diagonal step on the highlighted path represents the use of a common character (with that character's respective indices in the sequences highlighted in the margins).

13.4 Text Compression and the Greedy Method

In this section, we consider an important text-processing task, ***text compression***. In this problem, we are given a string X defined over some alphabet, such as the ASCII or Unicode character sets, and we want to efficiently encode X into a small binary string Y (using only the characters 0 and 1). Text compression is useful in any situation where we wish to reduce bandwidth for digital communications, so as to minimize the time needed to transmit our text. Likewise, text compression is useful for storing large documents more efficiently, so as to allow a fixed-capacity storage device to contain as many documents as possible.

The method for text compression explored in this section is the ***Huffman code***. Standard encoding schemes, such as ASCII, use fixed-length binary strings to encode characters (with 7 or 8 bits in the traditional or extended ASCII systems, respectively). The Unicode system was originally proposed as a 16-bit fixed-length representation, although common encodings reduce the space usage by allowing common groups of characters, such as those from the ASCII system, with fewer bits. The Huffman code saves space over a fixed-length encoding by using short code-word strings to encode high-frequency characters and long code-word strings to encode low-frequency characters. Furthermore, the Huffman code uses a variable-length encoding specifically optimized for a given string X over any alphabet. The optimization is based on the use of character ***frequencies***, where we have, for each character c, a count $f(c)$ of the number of times c appears in the string X.

To encode the string X, we convert each character in X to a variable-length code-word, and we concatenate all these code-words in order to produce the encoding Y for X. In order to avoid ambiguities, we insist that no code-word in our encoding be a prefix of another code-word in our encoding. Such a code is called a ***prefix code***, and it simplifies the decoding of Y to retrieve X. (See Figure 13.9.) Even with this restriction, the savings produced by a variable-length prefix code can be significant, particularly if there is a wide variance in character frequencies (as is the case for natural language text in almost every written language).

Huffman's algorithm for producing an optimal variable-length prefix code for X is based on the construction of a binary tree T that represents the code. Each edge in T represents a bit in a code-word, with an edge to a left child representing a "0" and an edge to a right child representing a "1." Each leaf v is associated with a specific character, and the code-word for that character is defined by the sequence of bits associated with the edges in the path from the root of T to v. (See Figure 13.9.) Each leaf v has a ***frequency***, $f(v)$, which is simply the frequency in X of the character associated with v. In addition, we give each internal node v in T a frequency, $f(v)$, that is the sum of the frequencies of all the leaves in the subtree rooted at v.

(a)

Character		a	b	d	e	f	h	i	k	n	o	r	s	t	u	v
Frequency	9	5	1	3	7	3	1	1	1	4	1	5	1	2	1	1

(b)

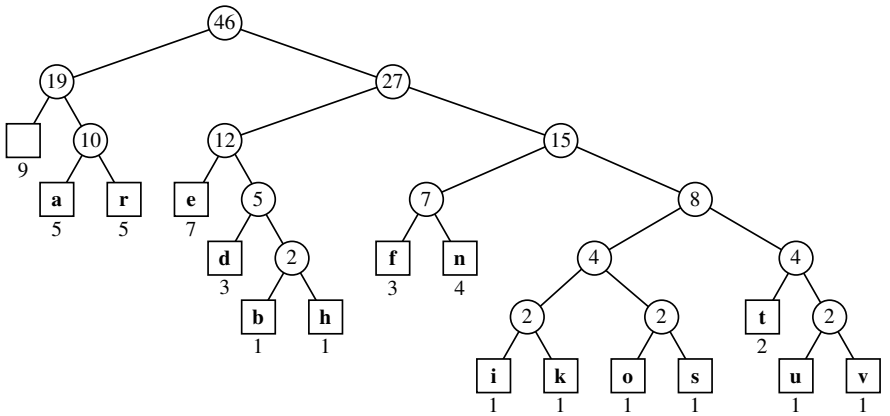


Figure 13.9: An illustration of an example Huffman code for the input string $X =$ `"a fast runner need never be afraid of the dark"`: (a) frequency of each character of X; (b) Huffman tree T for string X. The code for a character c is obtained by tracing the path from the root of T to the leaf where c is stored, and associating a left child with 0 and a right child with 1. For example, the code for "r" is 011, and the code for "h" is 10111.

13.4.1 The Huffman Coding Algorithm

The Huffman coding algorithm begins with each of the d distinct characters of the string X to encode being the root node of a single-node binary tree. The algorithm proceeds in a series of rounds. In each round, the algorithm takes the two binary trees with the smallest frequencies and merges them into a single binary tree. It repeats this process until only one tree is left. (See Code Fragment 13.8.)

Each iteration of the **while** loop in Huffman's algorithm can be implemented in $O(\log d)$ time using a priority queue represented with a heap. In addition, each iteration takes two nodes out of Q and adds one in, a process that will be repeated $d-1$ times before exactly one node is left in Q. Thus, this algorithm runs in $O(n+d\log d)$ time. Although a full justification of this algorithm's correctness is beyond our scope here, we note that its intuition comes from a simple idea—any optimal code can be converted into an optimal code in which the code-words for the two lowest-frequency characters, a and b, differ only in their last bit. Repeating the argument for a string with a and b replaced by a character c, gives the following:

Proposition 13.5: *Huffman's algorithm constructs an optimal prefix code for a string of length n with d distinct characters in $O(n+d\log d)$ time.*

Algorithm Huffman(X):
Input: String X of length n with d distinct characters
Output: Coding tree for X
Compute the frequency $f(c)$ of each character c of X.
Initialize a priority queue Q.
for each character c in X **do**
  Create a single-node binary tree T storing c.
  Insert T into Q with key $f(c)$.
while len(Q) > 1 **do**
  $(f_1, T_1) = Q$.remove_min()
  $(f_2, T_2) = Q$.remove_min()
  Create a new binary tree T with left subtree T_1 and right subtree T_2.
  Insert T into Q with key $f_1 + f_2$.
$(f, T) = Q$.remove_min()
return tree T

Code Fragment 13.8: Huffman coding algorithm.

13.4.2 The Greedy Method

Huffman's algorithm for building an optimal encoding is an example application of an algorithmic design pattern called the ***greedy method***. This design pattern is applied to optimization problems, where we are trying to construct some structure while minimizing or maximizing some property of that structure.

The general formula for the greedy method pattern is almost as simple as that for the brute-force method. In order to solve a given optimization problem using the greedy method, we proceed by a sequence of choices. The sequence starts from some well-understood starting condition, and computes the cost for that initial condition. The pattern then asks that we iteratively make additional choices by identifying the decision that achieves the best cost improvement from all of the choices that are currently possible. This approach does not always lead to an optimal solution.

But there are several problems that it does work for, and such problems are said to possess the ***greedy-choice*** property. This is the property that a global optimal condition can be reached by a series of locally optimal choices (that is, choices that are each the current best from among the possibilities available at the time), starting from a well-defined starting condition. The problem of computing an optimal variable-length prefix code is just one example of a problem that possesses the greedy-choice property.

13.5 Tries

The pattern-matching algorithms presented in Section 13.2 speed up the search in a text by preprocessing the pattern (to compute the failure function in the Knuth-Morris-Pratt algorithm or the last function in the Boyer-Moore algorithm). In this section, we take a complementary approach, namely, we present string searching algorithms that preprocess the text. This approach is suitable for applications where a series of queries is performed on a fixed text, so that the initial cost of preprocessing the text is compensated by a speedup in each subsequent query (for example, a Web site that offers pattern matching in Shakespeare's *Hamlet* or a search engine that offers Web pages on the *Hamlet* topic).

A ***trie*** (pronounced "try") is a tree-based data structure for storing strings in order to support fast pattern matching. The main application for tries is in information retrieval. Indeed, the name "trie" comes from the word "re*trie*val." In an information retrieval application, such as a search for a certain DNA sequence in a genomic database, we are given a collection S of strings, all defined using the same alphabet. The primary query operations that tries support are pattern matching and ***prefix matching***. The latter operation involves being given a string X, and looking for all the strings in S that contain X as a prefix.

13.5.1 Standard Tries

Let S be a set of s strings from alphabet Σ such that no string in S is a prefix of another string. A ***standard trie*** for S is an ordered tree T with the following properties (see Figure 13.10):

- Each node of T, except the root, is labeled with a character of Σ.
- The children of an internal node of T have distinct labels.
- T has s leaves, each associated with a string of S, such that the concatenation of the labels of the nodes on the path from the root to a leaf v of T yields the string of S associated with v.

Thus, a trie T represents the strings of S with paths from the root to the leaves of T. Note the importance of assuming that no string in S is a prefix of another string. This ensures that each string of S is uniquely associated with a leaf of T. (This is similar to the restriction for prefix codes with Huffman coding, as described in Section 13.4.) We can always satisfy this assumption by adding a special character that is not in the original alphabet Σ at the end of each string.

An internal node in a standard trie T can have anywhere between 1 and $|\Sigma|$ children. There is an edge going from the root r to one of its children for each character that is first in some string in the collection S. In addition, a path from the root of T to an internal node v at depth k corresponds to a k-character prefix $X[0:k]$

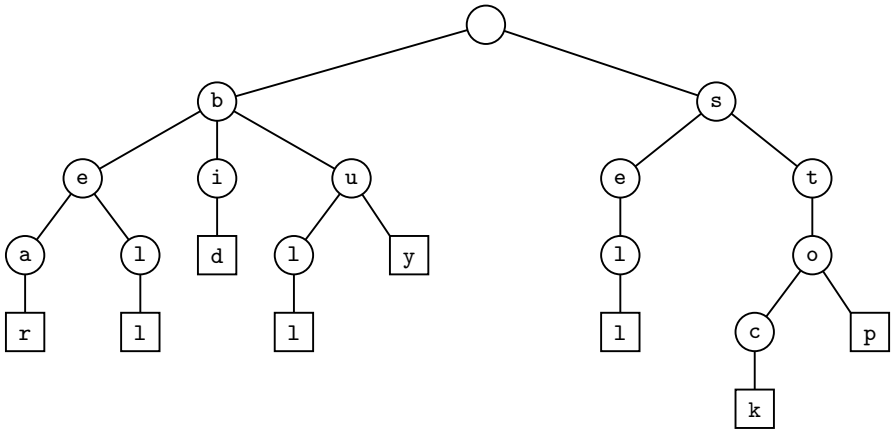


Figure 13.10: Standard trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}.

of a string X of S. In fact, for each character c that can follow the prefix $X[0:k]$ in a string of the set S, there is a child of v labeled with character c. In this way, a trie concisely stores the common prefixes that exist among a set of strings.

As a special case, if there are only two characters in the alphabet, then the trie is essentially a binary tree, with some internal nodes possibly having only one child (that is, it may be an improper binary tree). In general, although it is possible that an internal node has up to $|\Sigma|$ children, in practice the average degree of such nodes is likely to be much smaller. For example, the trie shown in Figure 13.10 has several internal nodes with only one child. On larger data sets, the average degree of nodes is likely to get smaller at greater depths of the tree, because there may be fewer strings sharing the common prefix, and thus fewer continuations of that pattern. Furthermore, in many languages, there will be character combinations that are unlikely to naturally occur.

The following proposition provides some important structural properties of a standard trie:

Proposition 13.6: *A standard trie storing a collection S of s strings of total length n from an alphabet Σ has the following properties:*

- *The height of T is equal to the length of the longest string in S.*
- *Every internal node of T has at most $|\Sigma|$ children.*
- *T has s leaves*
- *The number of nodes of T is at most $n+1$.*

The worst case for the number of nodes of a trie occurs when no two strings share a common nonempty prefix; that is, except for the root, all internal nodes have one child.

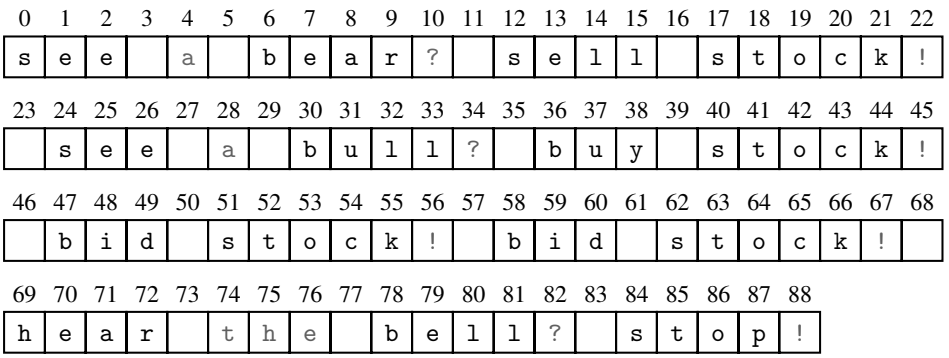
A trie T for a set S of strings can be used to implement a set or map whose keys are the strings of S. Namely, we perform a search in T for a string X by tracing down from the root the path indicated by the characters in X. If this path can be traced and terminates at a leaf node, then we know X is a key in the map. For example, in the trie in Figure 13.10, tracing the path for "bull" ends up at a leaf. If the path cannot be traced or the path can be traced but terminates at an internal node, then X is not a key in the map. In the example in Figure 13.10, the path for "bet" cannot be traced and the path for "be" ends at an internal node. Neither such word is in the map.

It is easy to see that the running time of the search for a string of length m is $O(m \cdot |\Sigma|)$, because we visit at most $m+1$ nodes of T and we spend $O(|\Sigma|)$ time at each node determining the child having the subsequent character as a label. The $O(|\Sigma|)$ upper bound on the time to locate a child with a given label is achievable, even if the children of a node are unordered, since there are at most $|\Sigma|$ children. We can improve the time spent at a node to be $O(\log |\Sigma|)$ or expected $O(1)$, by mapping characters to children using a secondary search table or hash table at each node, or by using a direct lookup table of size $|\Sigma|$ at each node, if $|\Sigma|$ is sufficiently small (as is the case for DNA strings). For these reasons, we typically expect a search for a string of length m to run in $O(m)$ time.

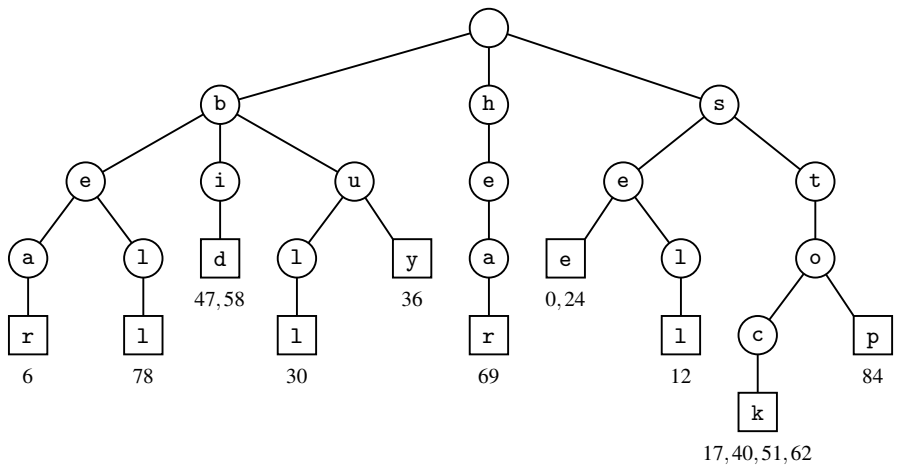
From the discussion above, it follows that we can use a trie to perform a special type of pattern matching, called ***word matching***, where we want to determine whether a given pattern matches one of the words of the text exactly. Word matching differs from standard pattern matching because the pattern cannot match an arbitrary substring of the text—only one of its words. To accomplish this, each word of the original document must be added to the trie. (See Figure 13.11.) A simple extension of this scheme supports prefix-matching queries. However, arbitrary occurrences of the pattern in the text (for example, the pattern is a proper suffix of a word or spans two words) cannot be efficiently performed.

To construct a standard trie for a set S of strings, we can use an incremental algorithm that inserts the strings one at a time. Recall the assumption that no string of S is a prefix of another string. To insert a string X into the current trie T, we trace the path associated with X in T, creating a new chain of nodes to store the remaining characters of X when we get stuck. The running time to insert X with length m is similar to a search, with worst-case $O(m \cdot |\Sigma|)$ performance, or expected $O(m)$ if using secondary hash tables at each node. Thus, constructing the entire trie for set S takes expected $O(n)$ time, where n is the total length of the strings of S.

There is a potential space inefficiency in the standard trie that has prompted the development of the ***compressed trie***, which is also known (for historical reasons) as the ***Patricia trie***. Namely, there are potentially a lot of nodes in the standard trie that have only one child, and the existence of such nodes is a waste. We discuss the compressed trie next.



(a)



(b)

Figure 13.11: Word matching with a standard trie: (a) text to be searched (articles and prepositions, which are also known as ***stop words***, excluded); (b) standard trie for the words in the text, with leaves augmented with indications of the index at which the given work begins in the text. For example, the leaf for the word `stock` notes that the word begins at indices 17, 40, 51, and 62 of the text.

13.5.2 Compressed Tries

A ***compressed trie*** is similar to a standard trie but it ensures that each internal node in the trie has at least two children. It enforces this rule by compressing chains of single-child nodes into individual edges. (See Figure 13.12.) Let T be a standard trie. We say that an internal node v of T is ***redundant*** if v has one child and is not the root. For example, the trie of Figure 13.10 has eight redundant nodes. Let us also say that a chain of $k \geq 2$ edges,

$$(v_0, v_1)(v_1, v_2) \cdots (v_{k-1}, v_k),$$

is ***redundant*** if:

- v_i is redundant for $i = 1, \ldots, k-1$.
- v_0 and v_k are not redundant.

We can transform T into a compressed trie by replacing each redundant chain $(v_0, v_1) \cdots (v_{k-1}, v_k)$ of $k \geq 2$ edges into a single edge (v_0, v_k), relabeling v_k with the concatenation of the labels of nodes $v_1, \ldots, v_k$.

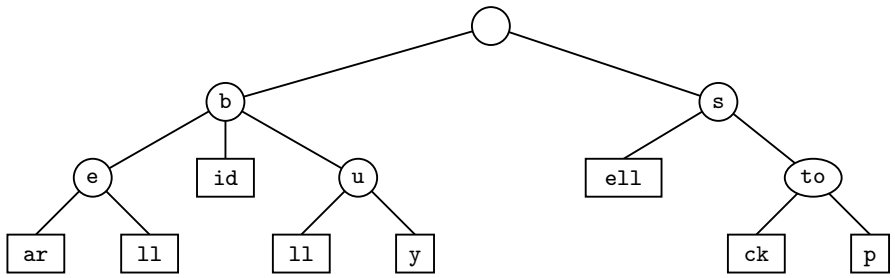


Figure 13.12: Compressed trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}. (Compare this with the standard trie shown in Figure 13.10.) In addition to compression at the leaves, notice the internal node with label `to` shared by words `stock` and `stop`.

Thus, nodes in a compressed trie are labeled with strings, which are substrings of strings in the collection, rather than with individual characters. The advantage of a compressed trie over a standard trie is that the number of nodes of the compressed trie is proportional to the number of strings and not to their total length, as shown in the following proposition (compare with Proposition 13.6).

Proposition 13.7: *A compressed trie storing a collection S of s strings from an alphabet of size d has the following properties:*

- *Every internal node of T has at least two children and most d children.*
- *T has s leaves nodes.*
- *The number of nodes of T is $O(s)$.*

The attentive reader may wonder whether the compression of paths provides any significant advantage, since it is offset by a corresponding expansion of the node labels. Indeed, a compressed trie is truly advantageous only when it is used as an ***auxiliary*** index structure over a collection of strings already stored in a primary structure, and is not required to actually store all the characters of the strings in the collection.

Suppose, for example, that the collection S of strings is an array of strings $S[0]$, $S[1]$, ..., $S[s-1]$. Instead of storing the label X of a node explicitly, we represent it implicitly by a combination of three integers $(i, j:k)$, such that $X = S[i][j:k]$; that is, X is the slice of $S[i]$ consisting of the characters from the jth up to but not including the kth. (See the example in Figure 13.13. Also compare with the standard trie of Figure 13.11.)

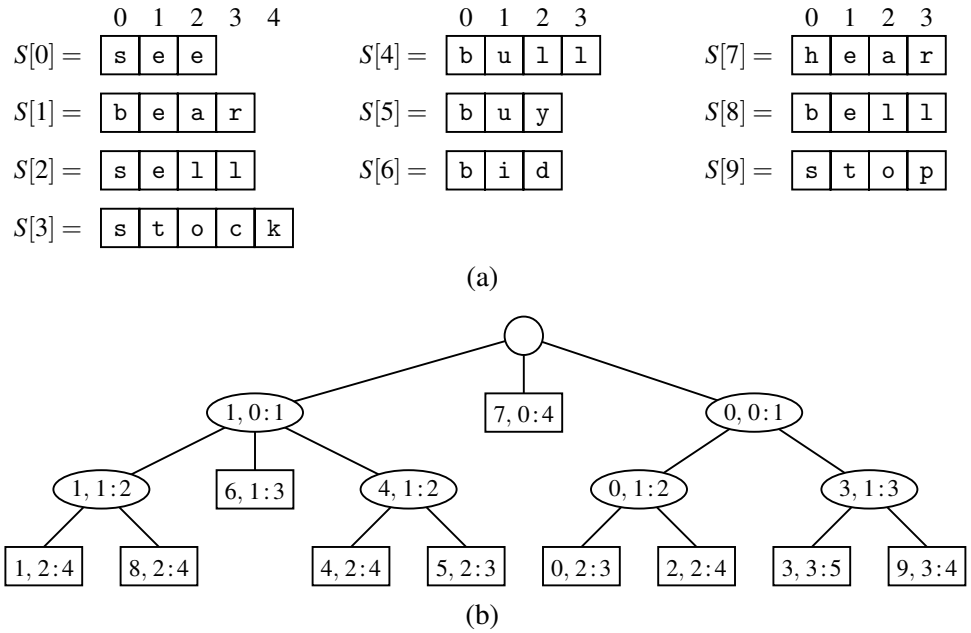


Figure 13.13: (a) Collection S of strings stored in an array. (b) Compact representation of the compressed trie for S.

This additional compression scheme allows us to reduce the total space for the trie itself from $O(n)$ for the standard trie to $O(s)$ for the compressed trie, where n is the total length of the strings in S and s is the number of strings in S. We must still store the different strings in S, of course, but we nevertheless reduce the space for the trie.

Searching in a compressed trie is not necessarily faster than in a standard tree, since there is still need to compare every character of the desired pattern with the potentially multi-character labels while traversing paths in the trie.

13.5.3 Suffix Tries

One of the primary applications for tries is for the case when the strings in the collection S are all the suffixes of a string X. Such a trie is called the ***suffix trie*** (also known as a ***suffix tree*** or ***position tree***) of string X. For example, Figure 13.14a shows the suffix trie for the eight suffixes of string "minimize." For a suffix trie, the compact representation presented in the previous section can be further simplified. Namely, the label of each vertex is a pair (j,k) indicating the string $X[j:k]$. (See Figure 13.14b.) To satisfy the rule that no suffix of X is a prefix of another suffix, we can add a special character, denoted with \$, that is not in the original alphabet Σ at the end of X (and thus to every suffix). That is, if string X has length n, we build a trie for the set of n strings $X[j:n]$, for $j = 0,\ldots,n-1$.

Saving Space

Using a suffix trie allows us to save space over a standard trie by using several space compression techniques, including those used for the compressed trie.

The advantage of the compact representation of tries now becomes apparent for suffix tries. Since the total length of the suffixes of a string X of length n is

$$1+2+\cdots+n = \frac{n(n+1)}{2},$$

storing all the suffixes of X explicitly would take $O(n^2)$ space. Even so, the suffix trie represents these strings implicitly in $O(n)$ space, as formally stated in the following proposition.

Proposition 13.8: *The compact representation of a suffix trie T for a string X of length n uses $O(n)$ space.*

Construction

We can construct the suffix trie for a string of length n with an incremental algorithm like the one given in Section 13.5.1. This construction takes $O(|\Sigma|n^2)$ time because the total length of the suffixes is quadratic in n. However, the (compact) suffix trie for a string of length n can be constructed in $O(n)$ time with a specialized algorithm, different from the one for general tries. This linear-time construction algorithm is fairly complex, however, and is not reported here. Still, we can take advantage of the existence of this fast construction algorithm when we want to use a suffix trie to solve other problems.

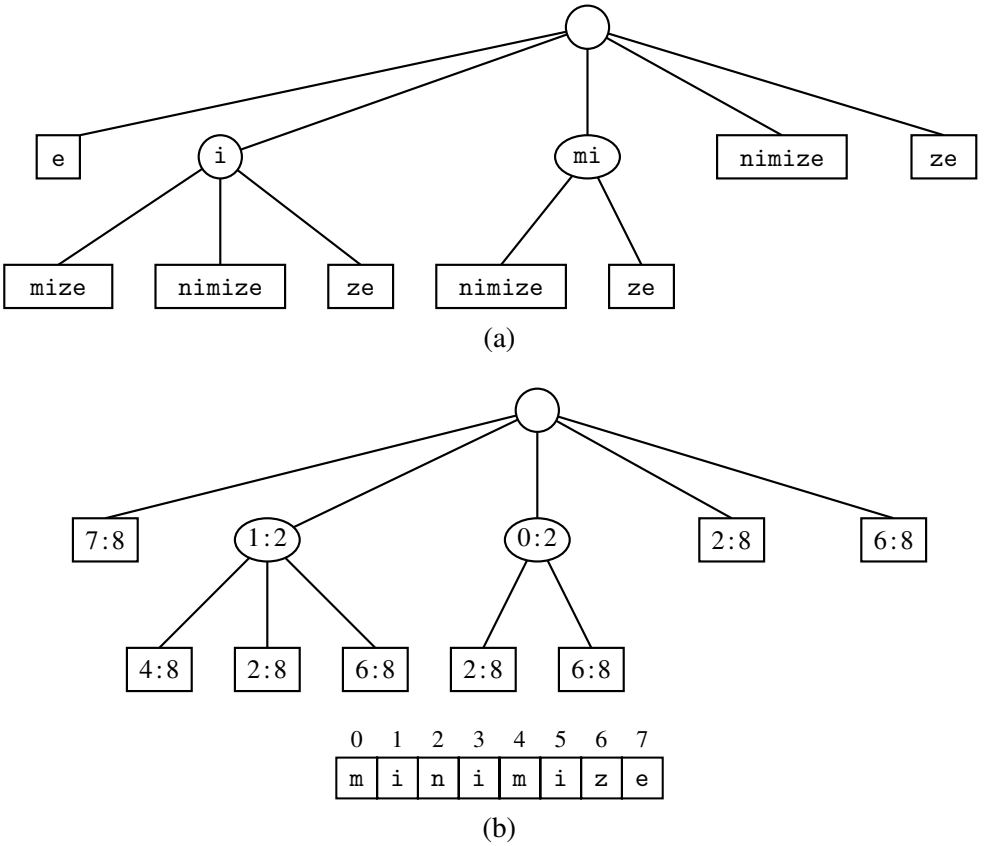


Figure 13.14: (a) Suffix trie T for the string $X =$ `"minimize"`. (b) Compact representation of T, where pair $j:k$ denotes slice $X[j:k]$ in the reference string.

Using a Suffix Trie

The suffix trie T for a string X can be used to efficiently perform pattern-matching queries on text X. Namely, we can determine whether a pattern P is a substring of X by trying to trace a path associated with P in T. P is a substring of X if and only if such a path can be traced. The search down the trie T assumes that nodes in T store some additional information, with respect to the compact representation of the suffix trie:

> If node v has label (j,k) and Y is the string of length y associated with the path from the root to v (included), then $X[k-y:k] = Y$.

This property ensures that we can easily compute the start index of the pattern in the text when a match occurs.

13.5.4 Search Engine Indexing

The World Wide Web contains a huge collection of text documents (Web pages). Information about these pages are gathered by a program called a ***Web crawler***, which then stores this information in a special dictionary database. A Web ***search engine*** allows users to retrieve relevant information from this database, thereby identifying relevant pages on the Web containing given keywords. In this section, we present a simplified model of a search engine.

Inverted Files

The core information stored by a search engine is a dictionary, called an ***inverted index*** or ***inverted file***, storing key-value pairs (w, L), where w is a word and L is a collection of pages containing word w. The keys (words) in this dictionary are called ***index terms*** and should be a set of vocabulary entries and proper nouns as large as possible. The elements in this dictionary are called ***occurrence lists*** and should cover as many Web pages as possible.

We can efficiently implement an inverted index with a data structure consisting of the following:

1. An array storing the occurrence lists of the terms (in no particular order).
2. A compressed trie for the set of index terms, where each leaf stores the index of the occurrence list of the associated term.

The reason for storing the occurrence lists outside the trie is to keep the size of the trie data structure sufficiently small to fit in internal memory. Instead, because of their large total size, the occurrence lists have to be stored on disk.

With our data structure, a query for a single keyword is similar to a word-matching query (Section 13.5.1). Namely, we find the keyword in the trie and we return the associated occurrence list.

When multiple keywords are given and the desired output are the pages containing ***all*** the given keywords, we retrieve the occurrence list of each keyword using the trie and return their intersection. To facilitate the intersection computation, each occurrence list should be implemented with a sequence sorted by address or with a map, to allow efficient set operations.

In addition to the basic task of returning a list of pages containing given keywords, search engines provide an important additional service by ***ranking*** the pages returned by relevance. Devising fast and accurate ranking algorithms for search engines is a major challenge for computer researchers and electronic commerce companies.

13.6 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-13.1 List the prefixes of the string $P =$"aaabbaaa" that are also suffixes of P.

R-13.2 What is the longest (proper) prefix of the string "cgtacgttcgtacg" that is also a suffix of this string?

R-13.3 Draw a figure illustrating the comparisons done by brute-force pattern matching for the text "aaabaadaabaaa" and pattern "aabaaa".

R-13.4 Repeat the previous problem for the Boyer-Moore algorithm, not counting the comparisons made to compute the $\mathsf{last}(c)$ function.

R-13.5 Repeat Exercise R-13.3 for the Knuth-Morris-Pratt algorithm, not counting the comparisons made to compute the failure function.

R-13.6 Compute a map representing the last function used in the Boyer-Moore pattern-matching algorithm for characters in the pattern string:

"the quick brown fox jumped over a lazy cat".

R-13.7 Compute a table representing the Knuth-Morris-Pratt failure function for the pattern string "cgtacgttcgtac".

R-13.8 What is the best way to multiply a chain of matrices with dimensions that are 10×5, 5×2, 2×20, 20×12, 12×4, and 4×60? Show your work.

R-13.9 In Figure 13.8, we illustrate that GTTTAA is a longest common subsequence for the given strings X and Y. However, that answer is not unique. Give another common subsequence of X and Y having length six.

R-13.10 Show the longest common subsequence array L for the two strings:

$$X = \text{"skullandbones"}$$
$$Y = \text{"lullabybabies"}$$

What is a longest common subsequence between these strings?

R-13.11 Draw the frequency array and Huffman tree for the following string:

"dogs do not spot hot pots or cats".

R-13.12 Draw a standard trie for the following set of strings:

{ abab, baba, ccccc, bbaaaa, caa, bbaacc, cbcc, cbca }.

R-13.13 Draw a compressed trie for the strings given in the previous problem.

R-13.14 Draw the compact representation of the suffix trie for the string:

"minimize minime".

Creativity

C-13.15 Describe an example of a text T of length n and a pattern P of length m such that force the brute-force pattern-matching algorithm achieves a running time that is $\Omega(nm)$.

C-13.16 Adapt the brute-force pattern-matching algorithm in order to implement a function, rfind_brute(T,P), that returns the index at which the *rightmost* occurrence of pattern P within text T, if any.

C-13.17 Redo the previous problem, adapting the Boyer-Moore pattern-matching algorithm appropriately to implement a function rfind_boyer_moore(T,P).

C-13.18 Redo Exercise C-13.16, adapting the Knuth-Morris-Pratt pattern-matching algorithm appropriately to implement a function rfind_kmp(T,P).

C-13.19 The count method of Python's str class reports the maximum number of *nonoverlapping* occurrences of a pattern within a string. For example, the call `'abababa'.count('aba')` returns 2 (not 3). Adapt the brute-force pattern-matching algorithm to implement a function, count_brute(T,P), with similar outcome.

C-13.20 Redo the previous problem, adapting the Boyer-Moore pattern-matching algorithm in order to implement a function count_boyer_moore(T,P).

C-13.21 Redo Exercise C-13.19, adapting the Knuth-Morris-Pratt pattern-matching algorithm appropriately to implement a function count_kmp(T,P).

C-13.22 Give a justification of why the compute_kmp_fail function (Code Fragment 13.4) runs in $O(m)$ time on a pattern of length m.

C-13.23 Let T be a text of length n, and let P be a pattern of length m. Describe an $O(n+m)$-time method for finding the longest prefix of P that is a substring of T.

C-13.24 Say that a pattern P of length m is a ***circular*** substring of a text T of length $n > m$ if P is a (normal) substring of T, or if P is equal to the concatenation of a suffix of T and a prefix of T, that is, if there is an index $0 \leq k < m$, such that $P = T[n-m+k:n] + T[0:k]$. Give an $O(n+m)$-time algorithm for determining whether P is a circular substring of T.

C-13.25 The Knuth-Morris-Pratt pattern-matching algorithm can be modified to run faster on binary strings by redefining the failure function as:

$$f(k) = \text{the largest } j < k \text{ such that } P[0:j]\widehat{p}_j \text{ is a suffix of } P[1:k+1],$$

where $\widehat{p}_j$ denotes the complement of the jth bit of P. Describe how to modify the KMP algorithm to be able to take advantage of this new failure function and also give a method for computing this failure function. Show that this method makes at most n comparisons between the text and the pattern (as opposed to the $2n$ comparisons needed by the standard KMP algorithm given in Section 13.2.3).

C-13.26 Modify the simplified Boyer-Moore algorithm presented in this chapter using ideas from the KMP algorithm so that it runs in $O(n+m)$ time.

C-13.27 Design an efficient algorithm for the matrix chain multiplication problem that outputs a fully parenthesized expression for how to multiply the matrices in the chain using the minimum number of operations.

C-13.28 A native Australian named Anatjari wishes to cross a desert carrying only a single water bottle. He has a map that marks all the watering holes along the way. Assuming he can walk k miles on one bottle of water, design an efficient algorithm for determining where Anatjari should refill his bottle in order to make as few stops as possible. Argue why your algorithm is correct.

C-13.29 Describe an efficient greedy algorithm for making change for a specified value using a minimum number of coins, assuming there are four denominations of coins (called quarters, dimes, nickels, and pennies), with values 25, 10, 5, and 1, respectively. Argue why your algorithm is correct.

C-13.30 Give an example set of denominations of coins so that a greedy change-making algorithm will not use the minimum number of coins.

C-13.31 In the ***art gallery guarding*** problem we are given a line L that represents a long hallway in an art gallery. We are also given a set $X = \{x_0, x_1, \ldots, x_{n-1}\}$ of real numbers that specify the positions of paintings in this hallway. Suppose that a single guard can protect all the paintings within distance at most 1 of his or her position (on both sides). Design an algorithm for finding a placement of guards that uses the minimum number of guards to guard all the paintings with positions in X.

C-13.32 Let P be a convex polygon, a ***triangulation*** of P is an addition of diagonals connecting the vertices of P so that each interior face is a triangle. The ***weight*** of a triangulation is the sum of the lengths of the diagonals. Assuming that we can compute lengths and add and compare them in constant time, give an efficient algorithm for computing a minimum-weight triangulation of P.

C-13.33 Let T be a text string of length n. Describe an $O(n)$-time method for finding the longest prefix of T that is a substring of the reversal of T.

C-13.34 Describe an efficient algorithm to find the longest palindrome that is a suffix of a string T of length n. Recall that a ***palindrome*** is a string that is equal to its reversal. What is the running time of your method?

C-13.35 Given a sequence $S = (x_0, x_1, \ldots, x_{n-1})$ of numbers, describe an $O(n^2)$-time algorithm for finding a longest subsequence $T = (x_{i_0}, x_{i_1}, \ldots, x_{i_{k-1}})$ of numbers, such that $i_j < i_{j+1}$ and $x_{i_j} > x_{i_{j+1}}$. That is, T is a longest decreasing subsequence of S.

C-13.36 Give an efficient algorithm for determining if a pattern P is a subsequence (not substring) of a text T. What is the running time of your algorithm?

C-13.37 Define the ***edit distance*** between two strings X and Y of length n and m, respectively, to be the number of edits that it takes to change X into Y. An edit consists of a character insertion, a character deletion, or a character replacement. For example, the strings `"algorithm"` and `"rhythm"` have edit distance 6. Design an $O(nm)$-time algorithm for computing the edit distance between X and Y.

C-13.38 Let X and Y be strings of length n and m, respectively. Define $B(j,k)$ to be the length of the longest common substring of the suffix $X[n-j:n]$ and the suffix $Y[m-k:m]$. Design an $O(nm)$-time algorithm for computing all the values of $B(j,k)$ for $j = 1,\ldots,n$ and $k = 1,\ldots,m$.

C-13.39 Anna has just won a contest that allows her to take n pieces of candy out of a candy store for free. Anna is old enough to realize that some candy is expensive, while other candy is relatively cheap, costing much less. The jars of candy are numbered 0, 1, …, $m-1$, so that jar j has n_j pieces in it, with a price of c_j per piece. Design an $O(n+m)$-time algorithm that allows Anna to maximize the value of the pieces of candy she takes for her winnings. Show that your algorithm produces the maximum value for Anna.

C-13.40 Let three integer arrays, A, B, and C, be given, each of size n. Given an arbitrary integer k, design an $O(n^2 \log n)$-time algorithm to determine if there exist numbers, a in A, b in B, and c in C, such that $k = a+b+c$.

C-13.41 Give an $O(n^2)$-time algorithm for the previous problem.

C-13.42 Given a string X of length n and a string Y of length m, describe an $O(n+m)$-time algorithm for finding the longest prefix of X that is a suffix of Y.

C-13.43 Give an efficient algorithm for deleting a string from a standard trie and analyze its running time.

C-13.44 Give an efficient algorithm for deleting a string from a compressed trie and analyze its running time.

C-13.45 Describe an algorithm for constructing the compact representation of a suffix trie, given its noncompact representation, and analyze its running time.

Projects

P-13.46 Use the LCS algorithm to compute the best sequence alignment between some DNA strings, which you can get online from GenBank.

P-13.47 Write a program that takes two character strings (which could be, for example, representations of DNA strands) and computes their edit distance, showing the corresponding pieces. (See Exercise C-13.37.)

P-13.48 Perform an experimental analysis of the efficiency (number of character comparisons performed) of the brute-force and KMP pattern-matching algorithms for varying-length patterns.

P-13.49 Perform an experimental analysis of the efficiency (number of character comparisons performed) of the brute-force and Boyer-Moore pattern-matching algorithms for varying-length patterns.

P-13.50 Perform an experimental comparison of the relative speeds of the brute-force, KMP, and Boyer-Moore pattern-matching algorithms. Document the relative running times on large text documents that are then searched using varying-length patterns.

P-13.51 Experiment with the efficiency of the `find` method of Python's `str` class and develop a hypothesis about which pattern-matching algorithm it uses. Try using inputs that are likely to cause both best-case and worst-case running times for various algorithms. Describe your experiments and your conclusions.

P-13.52 Implement a compression and decompression scheme that is based on Huffman coding.

P-13.53 Create a class that implements a standard trie for a set of ASCII strings. The class should have a constructor that takes a list of strings as an argument, and the class should have a method that tests whether a given string is stored in the trie.

P-13.54 Create a class that implements a compressed trie for a set of ASCII strings. The class should have a constructor that takes a list of strings as an argument, and the class should have a method that tests whether a given string is stored in the trie.

P-13.55 Create a class that implements a prefix trie for an ASCII string. The class should have a constructor that takes a string as an argument, and a method for pattern matching on the string.

P-13.56 Implement the simplified search engine described in Section 13.5.4 for the pages of a small Web site. Use all the words in the pages of the site as index terms, excluding stop words such as articles, prepositions, and pronouns.

P-13.57 Implement a search engine for the pages of a small Web site by adding a page-ranking feature to the simplified search engine described in Section 13.5.4. Your page-ranking feature should return the most relevant pages first. Use all the words in the pages of the site as index terms, excluding stop words, such as articles, prepositions, and pronouns.

Chapter Notes

The KMP algorithm is described by Knuth, Morris, and Pratt in their journal article [66], and Boyer and Moore describe their algorithm in a journal article published the same year [18]. In their article, however, Knuth *et al.* [66] also prove that the Boyer-Moore algorithm runs in linear time. More recently, Cole [27] shows that the Boyer-Moore algorithm makes at most $3n$ character comparisons in the worst case, and this bound is tight. All of the algorithms discussed above are also discussed in the book chapter by Aho [4], albeit in a more theoretical framework, including the methods for regular-expression pattern matching. The reader interested in further study of string pattern-matching algorithms is referred to the book by Stephen [90] and the book chapters by Aho [4], and Crochemore and Lecroq [30].

Dynamic programming was developed in the operations research community and formalized by Bellman [13].

The trie was invented by Morrison [79] and is discussed extensively in the classic *Sorting and Searching* book by Knuth [65]. The name "Patricia" is short for "Practical Algorithm to Retrieve Information Coded in Alphanumeric" [79]. McCreight [73] shows how to construct suffix tries in linear time. An introduction to the field of information retrieval, which includes a discussion of search engines for the Web, is provided in the book by Baeza-Yates and Ribeiro-Neto [8].